Decorator Design Pattern

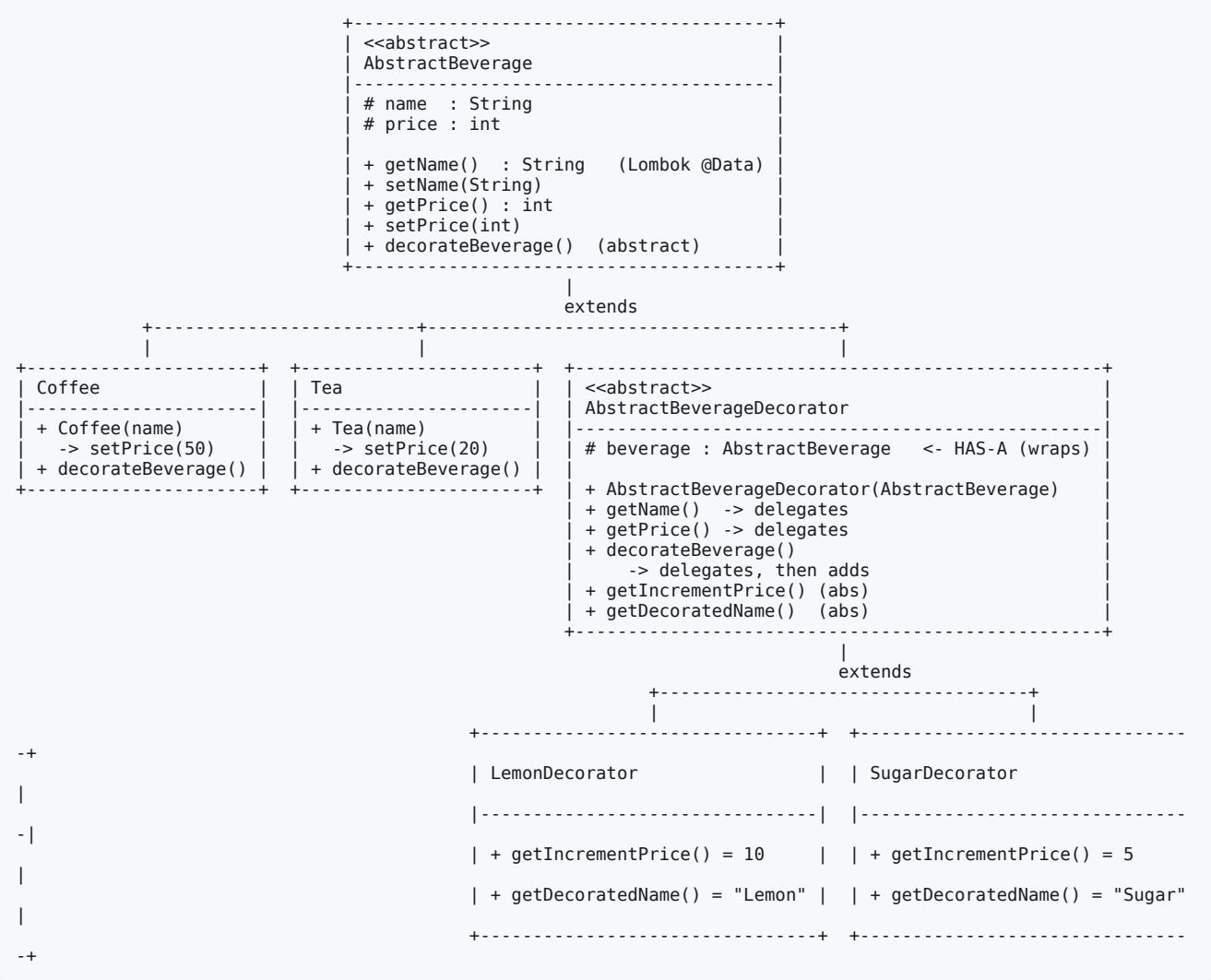
Intent

Attach additional responsibilities to an object dynamically. Decorators provide a flexible alternative to subclassing for extending functionality.

A decorator **is-a** component — it extends the same abstract type as the object it wraps — and simultaneously **has-a** component — it holds a reference to a wrapped instance of that same abstraction. Because a decorated object is still exactly the abstraction it decorates, wrappers can be composed at runtime, in any order, to any depth, by constructor chaining alone: `new SugarDecorator(new LemonDecorator(new Tea("Assam Tea")))`. No new subclass is required for any combination of add-ons.

This is the **canonical GoF Decorator**: an abstract `AbstractBeverage` component, two concrete components (`Coffee`, `Tea`), an abstract `AbstractBeverageDecorator` that both extends `AbstractBeverage` and wraps an `AbstractBeverage`, and two concrete decorators (`LemonDecorator`, `SugarDecorator`) that each add one unit of behavior/price and delegate everything else to the object they wrap.

UML class diagram



`AbstractBeverageDecorator` sits at the hinge of the diagram: it is drawn both as a subtype of `AbstractBeverage` (the *is-a* arrow going up) and as holding an `AbstractBeverage` field (the *has-a* relationship inside its own box). That dual relationship is the entire mechanism of the pattern — everything else in this module follows from it.

The players

```
component/AbstractBeverage                  the Component — abstract base every beverage shares
component/concreteComponent/Coffee          a ConcreteComponent — base beverage, price 50
component/concreteComponent/Tea             a ConcreteComponent — base beverage, price 20
decorator/AbstractBeverageDecorator         the Decorator — abstract wrapper, is-a and has-a AbstractBeverage
decorator/concreteDecorator/LemonDecorator  a ConcreteDecorator — adds "Lemon", +10
decorator/concreteDecorator/SugarDecorator  a ConcreteDecorator — adds "Sugar", +5
DecoratorDesignPattern                      the Client — composes decorators around components and runs them
```

GoF role	Class in this module
Component (common abstraction)	`AbstractBeverage` — `name`, `price`, `decorateBeverage()`
ConcreteComponent (base objects)	`Tea` (price 20), `Coffee` (price 50)
Decorator (abstract wrapper)	`AbstractBeverageDecorator` — extends `AbstractBeverage`, holds an `AbstractBeverage`
ConcreteDecorator (add-ons)	`SugarDecorator` (+5, "Sugar"), `LemonDecorator` (+10, "Lemon")
Client	`DecoratorDesignPattern.main()`

The code, line by line

`AbstractBeverage` — the Component

```
package com.design.patterns.component;

import lombok.Data;

@Data
public abstract class AbstractBeverage {
    protected String name;
    protected int price;

    public AbstractBeverage() {
    }

    public AbstractBeverage(String name) {
        this.name = name;
    }

    public abstract void decorateBeverage();

}
```

- `package com.design.patterns.component;` — the Component lives in its own package, separate from the decorators that will extend it, mirroring the GoF structure diagram (Component and Decorator are drawn as siblings, not parent/child packages).
- `@Data` (Lombok) — generates `getName()`, `setName(String)`, `getPrice()`, `setPrice(int)`, plus `equals()` / `hashCode()` / `toString()`, at compile time. This is why no getter/setter is hand-written anywhere in this file, yet `AbstractBeverageDecorator` can still `@Override public String getName()` below — Lombok's generated methods are ordinary, overridable instance methods.
- `protected String name;` / `protected int price;` — `protected` (not `private`) so that `Coffee` / `Tea` can read `name` / `price` directly by field access inside their own `decorateBeverage()` (see below), while Lombok's generated getters/setters give every other class in the module the same access through method calls.
- `public AbstractBeverage()` — a no-arg constructor. Present so the class is a normal bean-shaped Lombok target; it is not called anywhere in this module (both `Coffee` and `Tea` use the one-arg constructor), but keeping it avoids `AbstractBeverage` being unconstructable if a future test or framework needs a no-arg form.
- `public AbstractBeverage(String name)` — the constructor every concrete component actually uses: it seeds `name` and leaves `price` to be set separately (both `Coffee` and `Tea` call `setPrice(...)` in their own constructor bodies, right after `super(name)`).
- `public abstract void decorateBeverage();` — the one behavior every beverage (decorated or not) must be able to perform: print its own running description. Declaring it `abstract` here, rather than giving `AbstractBeverage` a default implementation, is what forces every concrete component *and* every decorator to participate in the delegation chain explained below — there is no "do nothing" fallback to fall back on.

`Coffee` / `Tea` — the ConcreteComponents

```
package com.design.patterns.component.concreteComponent;

import com.design.patterns.component.AbstractBeverage;
```

```
public class Coffee extends AbstractBeverage {

    public Coffee(String name) {
        super(name);
        setPrice(50);
    }

    @Override
    public void decorateBeverage() {
        System.out.println("The cost of " + name + ":" + price);
    }

}
```

- `package com.design.patterns.component.concreteComponent;` — a subpackage of `component`, holding only the concrete leaves of the Component hierarchy. Keeping ConcreteComponents out of the `component` package itself (rather than dropping `Coffee` / `Tea` alongside `AbstractBeverage`) makes the package structure self-documenting: one look at the directory tree shows which class is the abstraction and which are its concrete instances.
- `public Coffee(String name) { super(name); setPrice(50); }` — calls the `AbstractBeverage(String name)` constructor to set the name, then calls the Lombok-generated `setPrice(int)` to fix Coffee's price at 50. This is a plain field assignment through Lombok's setter — no decoration is involved yet, because at this point `Coffee` has no wrapper around it.
- `decorateBeverage()` reads `name` and `price` as bare (inherited, `protected`) fields, not through `getName()` / `getPrice()`. This matters once decorators enter the picture: a `Coffee` instance's own `decorateBeverage()` will only ever print *its own* base name and price, never a decorated name/price, because it never calls the overridable getters that a decorator might have overridden. `Tea` is identical except it fixes `price` at 20.

`AbstractBeverageDecorator` — the Decorator

```
package com.design.patterns.decorator;

import com.design.patterns.component.AbstractBeverage;

public abstract class AbstractBeverageDecorator extends AbstractBeverage {

    protected final AbstractBeverage beverage;

    public AbstractBeverageDecorator(AbstractBeverage beverage) {
        this.beverage = beverage;
    }

    @Override
    public String getName() {
        return beverage.getName() + ":" + getDecoratedName();
    }

    @Override
    public int getPrice() {
        return beverage.getPrice() + getIncrementPrice();
    }

    @Override
    public void decorateBeverage() {
        beverage.decorateBeverage();
        System.out.println("Added " + getDecoratedName() + " to " + beverage.getName()
                + " -> cost of " + getName() + ":" + getPrice());
    }

    public abstract int getIncrementPrice();

    public abstract String getDecoratedName();

}
```

This is the class that makes the pattern work. Every other class in the module is either something to be wrapped or a one-line specialization of this one.

- `public abstract class AbstractBeverageDecorator extends AbstractBeverage` — the **is-a** half of the pattern: because `AbstractBeverageDecorator` extends `AbstractBeverage`, a decorated beverage is still, statically and at runtime, an `AbstractBeverage`. This is what allows `new SugarDecorator(new LemonDecorator(new Tea(...)))` to type-check — `LemonDecorator` produces an `AbstractBeverage`, which is exactly what `SugarDecorator`'s constructor accepts, so decorators nest without any special-casing.
- `protected final AbstractBeverage beverage;` — the **has-a** half: a reference to the wrapped object. `final` because a decorator's target never changes after construction — swapping what a decorator wraps would silently rewrite the running order of the whole chain, so the field is locked down.
- `public AbstractBeverageDecorator(AbstractBeverage beverage) { this.beverage = beverage; }` — note this constructor does **not** call `super(name)` or touch `name` / `price` at all. A decorator has no name or price of its own to store; both are computed on demand (next two methods), so there is nothing to initialize on the `AbstractBeverage` side.

- `@Override public String getName()` — returns `beverage.getName() + ":" + getDecoratedName()`. This is **delegation, not copying**: it asks the wrapped object for its name (which — if the wrapped object is itself a decorator — recursively asks *its* wrapped object, and so on down to the base `Coffee` / `Tea`) and appends this layer's own suffix. No decorator ever stores a merged name string; the full name is reconstructed by walking the chain every time `getName()` is called.
- `@Override public int getPrice()` — the same delegation shape for price: wrapped price plus this layer's own increment, computed fresh on every call.
- `@Override public void decorateBeverage()` — first line is `beverage.decorateBeverage()`: **the wrapped object's own print/behavior runs before this layer adds anything.** This is what makes the base beverage's line ("The cost of Assam Tea:20") appear first in the output, followed by each wrapper's own line innermost-first — the decorator never skips or replaces the wrapped object's behavior, it always runs it and then augments it. The second line then prints this layer's addition, using the same `getName()` / `getPrice()` delegation described above so the printed name/price already include every inner layer.
- `public abstract int getIncrementPrice();` / `public abstract String getDecoratedName();` — the two extension points a concrete decorator must supply: how much this layer adds to the price, and what this layer calls itself. Everything else in `AbstractBeverageDecorator` — the delegation logic, the print statements — is written once, here, and shared by every concrete decorator without repetition.

`LemonDecorator` / `SugarDecorator` — the ConcreteDecorators

```
package com.design.patterns.decorator.concreteDecorator;

import com.design.patterns.component.AbstractBeverage;
import com.design.patterns.decorator.AbstractBeverageDecorator;

public class LemonDecorator extends AbstractBeverageDecorator {

    public LemonDecorator(AbstractBeverage beverage) {
        super(beverage);
    }

    @Override
    public int getIncrementPrice() {
        return 10;
    }

    @Override
    public String getDecoratedName() {
        return "Lemon";
    }

}
```

- `public LemonDecorator(AbstractBeverage beverage) { super(beverage); }` — takes any `AbstractBeverage` (a raw `Coffee` / `Tea`, or another decorator) and simply forwards it to `AbstractBeverageDecorator`'s constructor. This one line is the entire reason decorators are stackable: the constructor's parameter type is the abstraction, not a concrete class, so it accepts anything that is-a `AbstractBeverage`, including an already-decorated one.
- `getIncrementPrice()` returns `10`, `getDecoratedName()` returns `"Lemon"` — the only two facts specific to this decorator. `SugarDecorator` is structurally identical, returning `5` and `"Sugar"`. Neither class overrides `getName()`, `getPrice()`, or `decorateBeverage()` — they inherit that behavior unchanged from `AbstractBeverageDecorator`, because the delegation logic never varies between concrete decorators, only the increment and the label do.

`DecoratorDesignPattern` — the Client

```
package com.design.patterns;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

import com.design.patterns.component.AbstractBeverage;
import com.design.patterns.component.concreteComponent.Coffee;
import com.design.patterns.component.concreteComponent.Tea;
import com.design.patterns.decorator.concreteDecorator.LemonDecorator;
import com.design.patterns.decorator.concreteDecorator.SugarDecorator;

@SpringBootApplication
public class DecoratorDesignPattern {

    public static void main(String[] args) {
        SpringApplication.run(DecoratorDesignPattern.class, args);

        AbstractBeverage tea = new SugarDecorator(new LemonDecorator(new Tea("Assam Tea")));
        tea.decorateBeverage();

        AbstractBeverage coffee = new SugarDecorator(new LemonDecorator(new Coffee("Cappuccino")));
        coffee.decorateBeverage();
    }

}
```

- `@SpringBootApplication` / `SpringApplication.run(DecoratorDesignPattern.class, args)` — this module is a Maven submodule of a Spring Boot reactor (see `pom.xml`), so it boots as a Spring application for consistency with its sibling modules. Spring Boot itself plays no role in the Decorator pattern demonstrated below — the pattern is plain object composition, entirely independent of the framework.
- `AbstractBeverage tea = new SugarDecorator(new LemonDecorator(new Tea("Assam Tea")));` — construction happens inside-out: `new Tea("Assam Tea")` builds the base component (price 20), `new LemonDecorator(...)` wraps it (declaring `+10`/"Lemon"), and `new SugarDecorator(...)` wraps *that* (declaring `+5`/"Sugar"). The declared static type of `tea` is `AbstractBeverage` — the client never names a concrete decorator type once construction is finished, matching the Component/Decorator contract exactly.
- `tea.decorateBeverage();` — a single call that triggers the entire delegation chain described in `AbstractBeverageDecorator` above: `SugarDecorator` (outermost) delegates to `LemonDecorator`, which delegates to `Tea` (innermost), and each layer prints on the way back out.
- The `coffee` block repeats the same composition with `Coffee` (price 50) as the base, demonstrating that the same two decorator classes work unmodified against a different concrete component — no `Coffee`-specific or `Tea`-specific decorator code exists anywhere.

Why these design decisions

Why does `AbstractBeverageDecorator` both extend `AbstractBeverage` and hold an `AbstractBeverage`?

That dual relationship is the definition of Decorator, not an incidental design choice. The *is-a* half is what lets a decorated object be passed anywhere an `AbstractBeverage` is expected — including into another decorator's constructor, which is what makes stacking possible at all. The *has-a* half is what gives the decorator something to delegate to and augment. Drop either half and the pattern collapses: without *is-a*, decorators couldn't wrap each other; without *has-a*, there would be nothing to decorate.

Why is name/price computed by delegation instead of copied into the decorator at construction time?

Delegation (`beverage.getName() + ":" + getDecoratedName()`, recomputed on every call) keeps the wrapped object completely untouched and keeps every layer's contribution independently traceable. Copying the merged name/price into a field at construction time would work for a single read, but it would also mean the decorator's `getName()`/`getPrice()` stop reflecting the wrapped object if it could ever change after wrapping, and — more importantly for this codebase's own history (see below) — copying by calling overridable methods from inside a constructor is a well-known correctness trap in Java, because the subclass's override can run before the subclass has finished initializing its own state.

Why does `decorateBeverage()` call `beverage.decorateBeverage()` before printing its own line, rather than after or not at all?

Calling the wrapped object's behavior first, then augmenting, is what makes the printed output read as an onion unwrapping from the inside out — base beverage first, then each layer in the order it was applied. It also guarantees that **every** layer's behavior actually executes: a decorator that skipped this call would silently swallow the behavior of everything it wraps, which is precisely the bug this module's `History` section (below) records and fixes.

Why do `Coffee`/`Tea` print `name`/`price` as bare fields instead of calling `getName()`/`getPrice()`?

Because a `Coffee`/`Tea` instance's own `decorateBeverage()` must always describe *itself alone* — its own base name and base price — regardless of whether it is later wrapped by a decorator. If it called the overridable `getName()`/`getPrice()` instead, and a decorator later overrode those to include a suffix, then the base beverage's own print line would (incorrectly) start showing decorated values from inside a decorator's delegated call to `beverage.decorateBeverage()`. Reading the bare fields keeps the innermost print line stable no matter how many layers are stacked on top.

Why are there two decorators instead of one configurable decorator (e.g. constructed with a name/increment pair)?

The GoF Decorator pattern deliberately expresses each add-on as its own class rather than as configuration data, because that is what buys the Open/Closed benefit: adding a third topping later (e.g. `MilkDecorator`) means writing one new class with two one-line method overrides, with zero changes to `AbstractBeverage`, `Coffee`, `Tea`, or `AbstractBeverageDecorator`. A single parameterized decorator would avoid a class per topping, but it would also stop being an example of Decorator's actual mechanism — it would just be a builder with extra steps.

Execution flow (as run from `main`)

```
DecoratorDesignPattern.main
        │
        ├── SpringApplication.run(...)                    boots the app (the pattern itself doesn't need
Spring)
```

```
│
├── new Tea("Assam Tea")                          price = 20
│       └── new LemonDecorator(tea)               wraps Tea, declares +10 / "Lemon"
│               └── new SugarDecorator(lemonTea)      wraps LemonDecorator, declares +5 / "Sugar"
│
├── tea.decorateBeverage()        (tea is actually the outermost SugarDecorator)
│       └── SugarDecorator.decorateBeverage()   (inherited from AbstractBeverageDecorator)
│               ├── beverage.decorateBeverage()  → LemonDecorator.decorateBeverage()
│               │       ├── beverage.decorateBeverage() → Tea.decorateBeverage()
│               │       │       └── prints "The cost of Assam Tea:20"
│               │       └── prints "Added Lemon to Assam Tea -> cost of Assam Tea:Lemon:30"
│               │               (getName() = "Assam Tea:Lemon", getPrice() = 20+10 = 30)
│               └── prints "Added Sugar to Assam Tea:Lemon -> cost of Assam Tea:Lemon:Sugar:35"
│                       (getName() = "Assam Tea:Lemon:Sugar", getPrice() = 30+5 = 35)
│
└── same composition repeated for new Coffee("Cappuccino")   price = 50 → 60 → 65
```

Each layer's `getName()` / `getPrice()` recurses down to the base component and back up, so the printed name/price at every layer already include every layer beneath it — the trace above shows the running total (30, then 35) rather than each layer's isolated `+10` / `+5` contribution.

Expected output

Captured from a real run of the compiled, renamed class (`java -cp target/classes:<spring-boot-runtime-deps> com.design.patterns.DecoratorDesignPattern` , Spring Boot banner/logging lines omitted below — see **How to run**):

```
The cost of Assam Tea:20
Added Lemon to Assam Tea -> cost of Assam Tea:Lemon:30
Added Sugar to Assam Tea:Lemon -> cost of Assam Tea:Lemon:Sugar:35
The cost of Cappuccino:50
Added Lemon to Cappuccino -> cost of Cappuccino:Lemon:60
Added Sugar to Cappuccino:Lemon -> cost of Cappuccino:Lemon:Sugar:65
```

Every layer of the onion speaks in order — base beverage first, then each wrapper inside-out — because each decorator delegates before adding its own behavior. Prices: 20+10+5 = 35 correct, 50+10+5 = 65 correct.

How to run

This module is a Spring Boot submodule (`spring-boot-starter-web` on the classpath), so running the compiled class directly with only `target/classes` on the classpath fails with `NoClassDefFoundError: org/springframework/boot/SpringApplication` — the Spring Boot classes themselves live in separate dependency jars, not in `target/classes` . Two ways to get the real output:

```
# Build (JDK 11 required in this reactor — Lombok annotation processing breaks on newer JDKs here)
JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64 mvn -o clean compile

# Option 1: run with the full runtime classpath
JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64 mvn -o dependency:build-classpath -Dmdep.outputFile=/tmp/cp.txt
java -cp "target/classes:$(cat /tmp/cp.txt)" com.design.patterns.DecoratorDesignPattern

# Option 2: let Maven assemble the classpath for you
JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64 mvn -o spring-boot:run
```

Both options start an embedded Tomcat server (from `spring-boot-starter-web`) after the pattern demo prints its output, so the process keeps running until stopped (`Ctrl+C`) — the six lines under **Expected output** appear early in the log, before the "Tomcat started on port(s): 8080" line.

History

The first version accumulated name/price by copying state in the decorator constructor and never called `beverage.decorateBeverage()` — inner decorators' behavior was silently skipped ("Added Lemon..." never printed), and the demo in `main()` had the decorator lines commented out. Fixed: full delegation for `getName()` / `getPrice()` / `decorateBeverage()` , no overridable calls from constructors, and a live demo in `main()` .

A later cleanup pass renamed the runnable class from the generic `DesignPatternsApplication` to `DecoratorDesignPattern` (matching the sibling RealUseCase module's naming), corrected the misspelled `concretComponent` / `concretDecorator` packages to `concreteComponent` / `concreteDecorator` , and removed the vestigial default Spring Boot scaffold test (`DesignPatternsApplicationTests` , a `contextLoads()` test with no assertions about the pattern), for consistency with the sibling `Composite_Design_Pattern` and RealUseCase Decorator modules, neither of

which has a test directory. The output above was re-verified against the renamed, repackaged code and is unchanged from the original fix.